SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I KAVINMITHA.S(192511285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
kavinmitha .s

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System
A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.
Tasks:
(i) Design suitable intermediate code for the given assignment and Boolean expressions. *(K3)*
(ii) Explain how backpatching can help in handling conditional transaction validation. *(K4)*
(iii) Analyze the role of intermediate languages in improving banking software portability. *(K5)*

2. Smart Traffic Control System
An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.
Tasks:
(i) Construct intermediate code for the traffic signal control logic. *(K3)*
(ii) Explain how procedure calls improve modularity in the traffic control application. *(K4)*
(iii) Evaluate the importance of compiler optimization in real-time embedded systems. *(K5)*

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.
Tasks:
(i) Generate intermediate representation for the recommendation logic. *(K3)*
(ii) Discuss how case statements can efficiently handle product category selection. *(K4)*
(iii) Analyze the role of compiler design in improving large-scale e-commerce applications. *(K5)*

4. Hospital Patient Monitoring System
A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.
Tasks:
(i) Design intermediate code for patient condition monitoring logic. *(K3)*
(ii) Explain the significance of backpatching in emergency alert systems. *(K4)*
(iii) Evaluate how compiler optimization improves reliability in healthcare applications. *(K5)*

5. Online Airline Reservation System
An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.
Tasks:
(i) Develop intermediate code for ticket booking and seat allocation operations. *(K3)*

(ii) Explain how procedure calls support modular reservation management. *(K4)*
(iii) Analyze the role of intermediate languages in distributed airline reservation systems. *(K5)*

Set 2

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units.

Tasks:

(i) Design intermediate code for robotic movement and safety validation logic. *(K3)*
(ii) Explain how procedure calls improve modularity in industrial automation systems. *(K4)*
(iii) Analyze the role of compiler optimization in improving manufacturing efficiency. *(K5)*

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

(i) Generate intermediate representation for network threat detection logic. *(K3)*

(ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. *(K4)*
(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. *(K5)*

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

(i) Construct intermediate code for payment verification and fraud detection logic. *(K3)*
(ii) Explain how backpatching supports conditional transaction processing. *(K4)*
(iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. *(K5)*

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically. The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation.

Tasks:

(i) Develop intermediate code for irrigation control and environmental monitoring. *(K3)*
(ii) Explain the role of Boolean expressions in automated farming systems. *(K4)*

(iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. *(K5)*

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance.

Tasks:

(i) Design intermediate code for bandwidth allocation and quality adaptation logic. *(K3)*
(ii) Explain how case statements help manage multiple streaming quality levels. *(K4)*
(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. *(K5)*

Set 3

1. Autonomous Drone Delivery System

A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.

Tasks:

(i) Design intermediate code for drone navigation and obstacle detection logic. *(K3)*

(ii) Explain how Boolean expressions support autonomous decision-making in drones. *(K4)*

(iii) Analyze the importance of compiler optimization in real-time delivery systems. *(K5)*

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The compiler processes declarations, assignment statements, case statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

(i) Construct intermediate code for reservation and scheduling operations. *(K3)*

(ii) Discuss how case statements efficiently handle passenger ticket categories. *(K4)*

(iii) Evaluate the role of compiler optimization in large-scale transportation systems. *(K5)*

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:
(i) Generate intermediate code for load balancing and fault detection logic. *(K3)*
(ii) Explain the significance of backpatching in smart energy management systems. *(K4)*
(iii) Analyze how compiler optimization improves the performance of smart grids. *(K5)*

4. AI-Based Fraud Detection System
A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.
Tasks:
(i) Develop intermediate code for fraud detection and alert generation logic. *(K3)*
(ii) Explain how procedure calls improve modularity in fraud detection systems. *(K4)*
(iii) Evaluate the role of compiler design in secure financial applications. *(K5)*

5. Smart Warehouse Inventory Management System
A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.
Tasks:
(i) Design intermediate code for inventory tracking and stock update operations. *(K3)*
(ii) Explain how case statements help manage multiple product categories efficiently. *(K4)*
(iii) Analyze the importance of compiler optimization in warehouse automation systems. *(K5)*

Set 4

1. Smart Home Automation System

A home automation company is developing an intelligent control system that automatically manages lighting, temperature, and security devices. The compiler embedded in the controller processes declarations, assignment statements, and Boolean expressions to activate devices based on sensor inputs and user preferences. During real-time execution, inefficient intermediate code generation caused delays in device response and security alerts. The development team plans to redesign the compiler to improve automation efficiency and reliability.

Tasks:

(i) Design intermediate code for smart device control and sensor monitoring logic. *(K3)*

(ii) Explain how Boolean expressions support automated decision-making in smart homes. *(K4)*

(iii) Analyze the role of compiler optimization in improving home automation systems. *(K5)*

2. Online Examination Management System

An educational technology company is building an online examination platform that evaluates students automatically and manages exam scheduling. The compiler processes declarations, assignment statements, case statements, and procedure calls for student authentication and result generation. During large-scale online examinations, inefficient intermediate code execution caused delays in question loading and score processing. The organization plans to optimize the compiler to improve scalability and performance.

Tasks:

(i) Construct intermediate code for student authentication and evaluation logic. *(K3)*

(ii) Discuss how case statements help manage different examination categories. *(K4)*

(iii) Evaluate the importance of compiler optimization in large-scale online examination systems. *(K5)*

3. Intelligent Weather Forecasting System

A meteorological department is developing an intelligent weather forecasting application that analyzes environmental data in real time. The compiler processes assignment statements and Boolean expressions to monitor temperature, humidity, and rainfall conditions. During severe weather conditions, improper backpatching delayed alert generation and emergency notifications. The software engineers aim to redesign the compiler to improve prediction accuracy and system responsiveness.

Tasks:
(i) Generate intermediate code for weather monitoring and alert generation logic. *(K3)*
(ii) Explain the role of backpatching in handling emergency weather alerts. *(K4)*
(iii) Analyze how compiler optimization enhances real-time forecasting applications. *(K5)*

4. Automated Library Management System
A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.
Tasks:
(i) Develop intermediate code for book issue and return operations. *(K3)*
(ii) Explain how procedure calls improve modularity in library management systems. *(K4)*
(iii) Evaluate the significance of compiler optimization in educational information systems. *(K5)*

5. Industrial Water Quality Monitoring System
An environmental engineering company is developing a water quality monitoring system that continuously checks pH levels, contamination, and chemical composition in industrial plants. The compiler in the embedded controller evaluates Boolean expressions, assignment statements, and case statements for automated monitoring and alert generation. During critical contamination events, inefficient intermediate code execution delayed warning notifications and corrective actions. The company aims to redesign the compiler to improve reliability and response time.
Tasks:
(i) Design intermediate code for contamination detection and alert handling logic. *(K3)*
(ii) Explain how case statements efficiently manage different water quality conditions. *(K4)*
(iii) Analyze the importance of compiler optimization in industrial environmental monitoring systems. *(K5)*

Set 5

1. Smart Parking Management System

A smart city development company is designing an automated parking management system to monitor vehicle entry, slot allocation, and parking fee calculation. The compiler embedded in the control software processes declarations, assignment statements, Boolean expressions, and case statements to manage parking availability and security verification. During peak hours, inefficient intermediate code generation caused delays in slot assignment and payment confirmation. The company plans to redesign the compiler to improve real-time response and system efficiency.

Tasks:

(i) Design intermediate code for parking slot allocation and fee calculation logic. *(K3)*

(ii) Explain how Boolean expressions and case statements help manage parking operations efficiently. *(K4)*

(iii) Analyze the role of compiler optimization in improving smart parking management systems. *(K5)*

2. Digital Healthcare Appointment System

A healthcare organization is developing a digital appointment management platform to schedule doctor consultations and manage patient records. The compiler processes declarations, assignment statements, and procedure calls to handle appointment booking, cancellation, and patient verification. During high patient traffic, inefficient intermediate code execution caused delays in appointment confirmation and report generation. The development team plans to optimize the compiler for faster and more reliable healthcare services.

Tasks:

(i) Construct intermediate code for appointment scheduling and patient verification logic. *(K3)*

(ii) Explain how procedure calls improve modularity in healthcare management systems. *(K4)*

(iii) Evaluate the importance of compiler optimization in digital healthcare applications. *(K5)*

3. Automated Toll Collection System

A transportation authority is implementing an automated toll collection system that identifies vehicles and processes payments electronically. The compiler embedded in the toll management software evaluates Boolean expressions and assignment statements for vehicle classification and transaction validation. During festival seasons, improper backpatching caused delays in payment verification and traffic

management. The organization aims to redesign the compiler to improve transaction speed and operational efficiency.

Tasks:

(i) Generate intermediate code for vehicle identification and toll payment processing. *(K3)*

(ii) Explain the significance of backpatching in automated toll management systems. *(K4)*

(iii) Analyze how compiler optimization improves large-scale transportation applications. *(K5)*

4. Intelligent Fire Detection and Alarm System

A safety engineering company is developing an intelligent fire detection system that continuously monitors smoke and temperature sensors in industrial buildings. The compiler processes Boolean expressions, assignment statements, and case statements to activate alarms and emergency protocols. During emergency situations, inefficient intermediate code execution delayed warning notifications and evacuation procedures. The software engineers plan to optimize the compiler for faster real-time response.

Tasks:

(i) Develop intermediate code for fire detection and emergency alert logic. *(K3)*

(ii) Explain how case statements efficiently handle multiple emergency conditions. *(K4)*

(iii) Evaluate the role of compiler optimization in industrial safety systems. *(K5)*

5. AI-Based Customer Support Chatbot

A software company is building an AI-based customer support chatbot to handle user queries and provide automated responses. The compiler processes declarations, assignment statements, Boolean expressions, and procedure calls to analyze user requests and generate responses dynamically. During large-scale customer interactions, inefficient intermediate code generation caused delays in response delivery and reduced system performance. The organization plans to redesign the compiler to improve scalability and user experience.

Tasks:

(i) Design intermediate code for chatbot query processing and response generation. *(K3)*

(ii) Explain how procedure calls improve modularity in AI-based chatbot systems. *(K4)*

(iii) Analyze the importance of compiler optimization in intelligent customer support applications. *(K5)*

ANSWERS:

Q1. Smart Home Automation System

(i) Intermediate code for smart device control:

Example:

```
if (temperature > 30)
   AC = ON;
else
   AC = OFF;
```

Three-address code:

```
T1 = temperature > 30
IF T1 GOTO L1
GOTO L2
L1: AC = ON
GOTO L3
L2: AC = OFF
L3:
```

(ii) Boolean expressions:

Boolean expressions help the system make automatic decisions based on sensor values. For example, temperature > 30 AND humidity > 60 can activate the air conditioner.

(iii) Compiler optimization:

Compiler optimization reduces unnecessary instructions and execution time. It improves device response, reduces resource usage, and makes smart-home automation faster and more reliable.

Q2. Online Examination Management System

(i) Intermediate code for authentication and evaluation:

```
T1 = username == stored_user
T2 = password == stored_pass
T3 = T1 AND T2
```

```
IF T3 GOTO L1
GOTO L2

L1: result = calculate_marks()
L2: display_result()
```

(ii) Case statements:

Case statements can manage different examination categories efficiently, such as:

```
CASE exam_type
 1: Internal Exam
 2: Semester Exam
 3: Model Exam
ENDCASE
```

This avoids many separate conditional statements.

(iii) Compiler optimization:

Optimization reduces execution time and improves scalability. This is important when many students access the examination system simultaneously.

Q3. Intelligent Weather Forecasting System

(i) Intermediate code:

```
T1 = temperature > 40
T2 = humidity < 30
T3 = T1 AND T2
IF T3 GOTO L1
GOTO L2

L1: alert = "Heat Warning"
L2: display_weather()
```

(ii) Backpatching:

Backpatching fills the target addresses of conditional and unconditional jumps after the required labels are known. It helps generate correct control flow for emergency weather alerts.

(iii) Compiler optimization:

Optimization makes weather-processing programs execute faster and

reduces unnecessary computations. This allows alerts and forecasts to be generated quickly during severe weather.

Q4. Automated Library Management System

(i) Intermediate code:

```
T1 = book_available == TRUE
IF T1 GOTO L1
GOTO L2

L1: issue_book()
   available = available - 1
   fine = 0
   GOTO L3

L2: display("Book Not Available")

L3:
```

(ii) Procedure calls:
Procedure calls divide the library system into separate functions such as issue_book(), return_book(), verify_member(), and calculate_fine(). This improves modularity, code reuse, and maintenance.

(iii) Compiler optimization:
Optimization reduces execution time and improves the speed of book issue, return, member verification, and fine calculation operations. It makes the library system faster and more reliable.

Q5. Industrial Water Quality Monitoring System

(i) Intermediate code:

```
T1 = pH < 6
T2 = contamination == HIGH
T3 = T1 OR T2
IF T3 GOTO L1
GOTO L2

L1: alert = ON
```

```
   display("Water Quality Warning")
   GOTO L3

L2: alert = OFF

L3:
```

(ii) Case statements:
Case statements can classify different water-quality conditions efficiently.

```
CASE quality
 1: display("Safe")
 2: display("Warning")
 3: display("Critical")
ENDCASE
```

This makes the monitoring logic simple and organized.

(iii) Compiler optimization:
Compiler optimization reduces execution time and improves response speed. In industrial monitoring, this helps generate contamination warnings and corrective actions quickly and reliably.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

		requirements accurately			
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation